

MELON: a modern C++23 library for graph algorithms and network optimization

François Hamonic ¹ ¶

¹ Institut Méditerranéen de Biodiversité et d'Écologie marine et continentale (IMBE), Aix Marseille Univ, Avignon Université, CNRS, IRD, Marseille, France  ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

Graphs describe anything made of objects and pairwise relations: road and transit networks, habitat patches and the corridors between them, supply chains, meshes, state machines. The classical questions asked of them (which vertices are reachable, what is the shortest or most reliable route, how much flow a network can carry, what the cheapest way to route demand is) have well-known algorithms, and research code has to compute them quickly, on data that already lives in whatever structure the rest of the program uses.

MELON (Modern and Efficient Library for Optimization in Networks) is a header-only, dependency-free C++23 library of graph containers, lazy graph views and algorithms, built on C++20 concepts and ranges. It ships traversals (breadth-first and depth-first search, topological sort, strongly and weakly connected components), shortest paths (Dijkstra, A*, bidirectional Dijkstra, bi-objective Dijkstra, Bellman–Ford, network Voronoi), flows and trees (Edmonds–Karp, Dinitz, network simplex for minimum-cost flow, Kruskal), knapsack branch-and-bound and Bentley–Ottmann segment intersection, together with the heaps, flat maps and disjoint-set structures they are built on. Algorithms are constrained by concepts rather than written against one graph class, so they run on MELON's containers, on its zero-copy views, and on the user's own data structure when it exposes a handful of member or free functions.

Statement of need

C++ is the implementation language of choice when a graph computation is the inner loop of a larger optimization or simulation, but the two established generic C++ graph libraries each force a compromise. The Boost Graph Library ([Siek et al., 2002](#)) is generic, but its genericity predates language support for concepts: it is expressed through traits classes, tag dispatch and external property maps, and a mistake yields template diagnostics that are notoriously hard to read. LEMON ([Dezső et al., 2011](#)) is fast and comfortable, but it is unmaintained (its last release, 1.3.1, dates from 2014) and needs source patches to build under C++17 and later. In practice many research codes fall back on `std::vector<std::vector<int>>` and a hand-written Dijkstra, and pay for it later in correctness, performance and reuse.

MELON was written to keep LEMON's speed and ergonomics while expressing Boost.Graph's genericity with the tools C++20 and C++23 now provide. Its intended users are researchers and engineers who write performance-sensitive graph code in modern C++: operations research and computational ecology, where the author's own work originated, but also routing, network analysis and any domain where the same algorithm must run over a graph, its reverse and a filtered subgraph without duplicating memory, or over a structure the program already holds (a raster grid, a mesh, a compressed sparse row array coming out of a solver) without first copying it into a “real” graph.

41 State of the field

42 Boost.Graph and LEMON are the direct predecessors, and MELON's documentation carries
43 a translation table from both. Neither can adopt concepts and ranges without breaking a
44 decades-old interface, and LEMON is no longer developed, which is the “build rather than
45 contribute” argument for a new library. Two C++20 efforts are closer in spirit. NWGraph
46 (Lumsdaine et al., 2022) rebuilt a generic graph library on ranges of ranges, and the WG21
47 Graph Library proposal (Ratzloff & Lumsdaine, 2024) draws on it to define a graph container
48 interface for a future standard. Both define algorithms as functions that run to completion,
49 with visitors or output maps for intermediate results, and both target the standardization
50 process rather than a shipping toolkit; MELON explores a different point in the design space
51 (algorithms as steppable ranges, graph-owned maps, compile-time storage traits) and already
52 provides the network-flow and minimum-cost-flow algorithms an optimization user needs.
53 Analysis toolkits such as NetworkKit (Staudt et al., 2016), igraph (Csárdi & Nepusz, 2006)
54 and graph-tool (Peixoto, 2014) have compiled cores but are driven from Python or R over a
55 fixed internal representation; they are the right tool for interactive network analysis, not for
56 embedding a generic algorithm into a C++ program over the program's own types.

57 Software design

58 Five decisions shape the library, each visible at the call site.

59 **The interface is a set of concepts, not a base class.** `graph`, `outward_incidence_graph`,
60 `has_vertex_map<G, T>` and their relatives describe what a type must *do*, and every algorithm
61 states in its signature exactly the capabilities it needs. Instantiating an algorithm on an
62 unsuitable structure fails at the call site with a diagnostic naming the missing requirement.
63 The accessors the concepts are written against (`vertices`, `out_arcs`, `arc_target`, ...) are
64 customization point objects that accept a member function or a free function found by
65 argument-dependent lookup, so a type from another library can be adapted without touching
66 it. Several of them *synthesize* what is missing: a structure exposing `vertices`, `out_arcs` and
67 `arc_target` automatically gains `out_neighbors`, `arcs`, `arcs_entries`, `degrees` and `counts`,
68 and the synthesized range is given the strongest iterator category the primitives allow.

69 **The graph owns its data maps.** Instead of external property maps handed in by the caller, an
70 algorithm asks the graph for scratch storage through `create_vertex_map<T>(g)`, and states
71 that need as a constraint. The storage type is the graph's choice: MELON's containers hand
72 back a flat array, a graph with non-integral identifiers may hand back a hash map, and a view
73 can hand out storage that already exists. Maps are requested under a *role*, so two maps of the
74 same value type are never confused.

75 **Algorithms are steppable ranges.** An algorithm object exposes `finished()`, `current()` and
76 `advance()`, which makes it a standard input range: the loop body is the visitor, `break` is the
77 early exit, and `std::views::take` or `std::ranges::find_if` compose with it directly. Two
78 searches can be advanced in lockstep, which is exactly how the bidirectional and competing
79 Dijkstra variants are implemented. Algorithm objects are move-only, since copying a search
80 state is never the cheap operation the syntax suggests, and `reset()` reuses the allocated state
81 across queries.

82 **You do not pay for what you do not use.** Each algorithm takes a traits type selecting its heap,
83 its semiring and what it records. Optional state lives in `[[no_unique_address]]` members
84 that collapse to zero bytes when a flag is off, and the accessors that would read them are
85 removed from the overload set by a `requires` clause, so a misuse is a compile error rather
86 than a runtime check. Swapping the semiring turns Dijkstra into a most-reliable-path or a
87 widest-path search without touching the traversal.

88 **Views instead of copies.** `views::reverse`, `views::subgraph`, `views::undirect` and the `map`

providing views are lazy adaptors that satisfy the same concepts as containers, compose with a pipe operator, and follow the `std::ranges` ownership rules: an lvalue is referenced, an rvalue is owned, a view passes through unchanged.

The cost of these choices is a C++23 toolchain (GCC 14, Clang 18, Apple Clang 21 or MSVC 17.11 at minimum), longer compile times than a non-template library, and single-threaded algorithms. Preconditions are asserted rather than thrown, and the API outside `melon/detail/` and `melon/experimental/` is frozen for the 1.x series under semantic versioning.

Performance comes from the same design: no virtual dispatch or type erasure anywhere, a compressed adjacency container whose per-vertex arc ranges are consecutive integers, contiguous maps that let the shortest-path family issue explicit prefetches, array-indexed heap positions instead of hash lookups, and a branchless breadth-first search selected by `if constexpr` when the traits allow it. Figure 1 shows two of the measurements published in the companion benchmark repository (Hamonic, 2026a), where every library is built with the same compiler and every timed configuration must produce the same result digest before a chart is drawn.

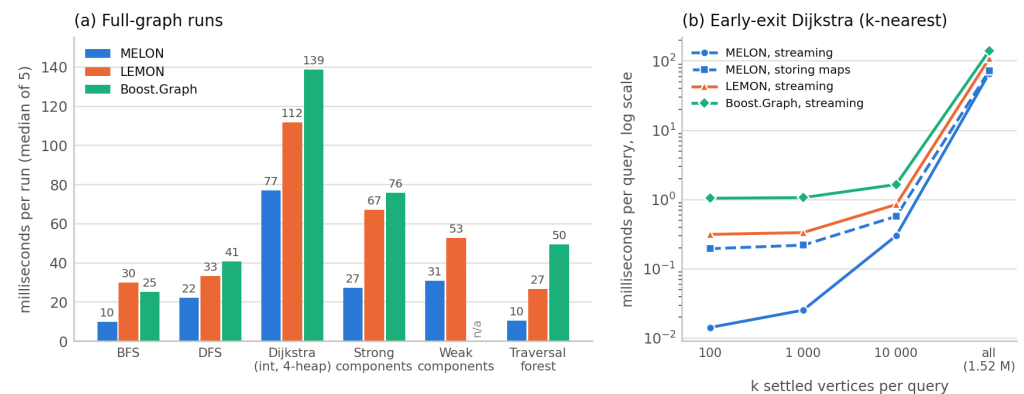


Figure 1: Runtime on the largest USA road network of the 9th DIMACS Implementation Challenge (Demetrescu et al., 2009) (USA-road-t.NE, 1.52 M vertices, 3.9 M arcs), each library in its fastest container, GCC 14.1, -O3, AMD Ryzen 7 7800X3D, median of five repetitions. (a) Full runs of six algorithms; Boost.Graph has no correct weak-components algorithm for directed graphs. (b) Cost of a query that consumes the k nearest vertices from a source and stops, with every library in the leanest configuration its API allows: because a MELON algorithm is a range that allocates only what its traits request, an early-exit query costs one to two orders of magnitude less than through a run-to-completion interface, and the advantage amortizes as the search grows to cover the graph.

Research impact statement

MELON grew out of the author's doctoral work on landscape connectivity optimization (Hamonic, 2023; Hamonic, Albert, et al., 2023; Hamonic, Couëtoux, et al., 2023), where the inner loop is a large number of shortest-path computations over habitat graphs, and it is the graph layer of GECOT, the Graph-based Ecological Connectivity Optimization Tool published in Methods in Ecology and Evolution (Hamonic et al., 2025): GECOT declares `melon/1.0.0` as a dependency and builds its landscape graphs, per-species maps and connectivity computations on MELON's containers, maps and shortest-path algorithms. A companion library by the same author for linear and mixed-integer programming (Hamonic, 2026b) shares its design and is used alongside it. The repository has been public since December 2021 with more than a thousand commits, is packaged on Conan Center, and has received issues and pull requests from external users, including a compiler-compatibility fix and tests for the topological-sort algorithm; the 1.0.0 release froze the public API after a redesign that removed every third-party dependency. The benchmark repository (Hamonic, 2026a) also serves as an independently reproducible comparison of LEMON, Boost.Graph and CGAL (The CGAL Project, 2024) on

118 DIMACS (Demetrescu et al., 2009) and SNAP (Leskovec & Krevl, 2014) instances, with
119 cross-library result validation, which is useful beyond MELON itself.

120 AI usage disclosure

121 This manuscript was drafted with the assistance of a generative AI coding assistant (Claude,
122 Anthropic), working from the repository, its documentation and the benchmark data; the
123 author reviewed, corrected and takes responsibility for every statement. [State here whether
124 generative AI tools were used in the development of the software itself or of its documentation,
125 and if so how their output was verified: for example, through the test suite, the sanitizer runs
126 and the cross-library benchmark validation.]

127 Acknowledgements

128 This work is grounded in the PhD thesis and postdoctoral positions of the author, funded by
129 Région Sud Provence-Alpes-Côte d'Azur, Natural Solutions, the European Research Council
130 grant SCALED to Cécile H. Albert (ERC-STG no. 949812), the ANR project RESILIENCE
131 (ANR-24-PEVD-0002) and the OASIS project of Aix-Marseille University's ITEM institute.
132 The author thanks the external contributors to the repository, in particular Simon Spoorendonk
133 for the GCC 14 build fix and GitHub user pradkrish for the topological-sort tests.

134 References

- 135 Csárdi, G., & Nepusz, T. (2006). The igraph software package for complex network research.
136 *InterJournal Complex Systems*, 1695. <https://igraph.org>
- 137 Demetrescu, C., Goldberg, A. V., & Johnson, D. S. (Eds.). (2009). *The shortest path problem:*
138 *Ninth DIMACS implementation challenge* (Vol. 74). American Mathematical Society.
139 <https://doi.org/10.1090/dimacs/074>
- 140 Dezső, B., Jüttner, A., & Kovács, P. (2011). LEMON – an open source C++ graph
141 template library. *Electronic Notes in Theoretical Computer Science*, 264(5), 23–45. <https://doi.org/10.1016/j.entcs.2011.06.003>
- 142
- 143 Hamonic, F. (2023). *Algorithms for conservation and restoration of landscape connectivity*
144 [PhD thesis, Aix-Marseille Université]. <https://theses.fr/2023AIXM9063>
- 145 Hamonic, F. (2026a). *Melon_benchmark: Runtime comparison of MELON against LEMON,*
146 *Boost.Graph and CGAL*. https://github.com/fhamonic/melon_benchmark
- 147 Hamonic, F. (2026b). *Mippp: Header-only C++ library for linear, mixed-integer and quadratic*
148 *programming*. <https://github.com/fhamonic/mippp>
- 149 Hamonic, F., Albert, C., Couëtoux, B., & Vaxès, Y. (2023). Optimizing the ecological
150 connectivity of landscapes. *Networks*, 81(2), 278–293. <https://doi.org/10.1002/net.22131>
- 151 Hamonic, F., Couëtoux, B., Vaxès, Y., & Albert, C. H. (2023). Cumulative effects on
152 habitat networks: How greedy should we be? *Biological Conservation*, 282, 110066.
153 <https://doi.org/10.1016/j.biocon.2023.110066>
- 154 Hamonic, F., Vaxès, Y., Couëtoux, B., & Albert, C. H. (2025). GECOT: Graph-based ecological
155 connectivity optimization tool. *Methods in Ecology and Evolution*, 16(9), 1914–1922.
156 <https://doi.org/10.1111/2041-210X.70055>
- 157 Leskovec, J., & Krevl, A. (2014). *SNAP datasets: Stanford large network dataset collection*.
158 <http://snap.stanford.edu/data>

- 159 Lumsdaine, A., D'Alessandro, L., Deweese, K., Firoz, J., Liu, X. T., McMillan, S., Ratzloff,
160 J. P., & Zalewski, M. (2022). NWGraph: A library of generic graph algorithms and
161 data structures in C++20. *36th European Conference on Object-Oriented Programming*
162 (*ECOOP 2022*), 222, 31:1–31:28. <https://doi.org/10.4230/LIPIcs.ECOOP.2022.31>
- 163 Peixoto, T. P. (2014). *The graph-tool python library*. figshare. [https://doi.org/10.6084/m9.](https://doi.org/10.6084/m9.figshare.1164194)
164 [figshare.1164194](https://doi.org/10.6084/m9.figshare.1164194)
- 165 Ratzloff, P., & Lumsdaine, A. (2024). *P1709R5: Graph library*. ISO/IEC JTC1/SC22/WG21,
166 the C++ Standards Committee. [https://www.open-std.org/jtc1/sc22/wg21/docs/papers/](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1709r5.pdf)
167 [2024/p1709r5.pdf](https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1709r5.pdf)
- 168 Siek, J. G., Lee, L.-Q., & Lumsdaine, A. (2002). *The Boost graph library: User guide and*
169 *reference manual*. Addison-Wesley. ISBN: 978-0-201-72914-6
- 170 Staudt, C. L., Sazonovs, A., & Meyerhenke, H. (2016). NetworkKit: A tool suite for large-scale
171 complex network analysis. *Network Science*, 4(4), 508–530. [https://doi.org/10.1017/nws.](https://doi.org/10.1017/nws.2016.20)
172 [2016.20](https://doi.org/10.1017/nws.2016.20)
- 173 The CGAL Project. (2024). *CGAL user and reference manual* (5.6.1 ed.). CGAL Editorial
174 Board. <https://doc.cgal.org/5.6.1/Manual/packages.html>

DRAFT